



UNIVERSITÀ DI PISA

Dipartimento di Ingegneria dell'Informazione
Corso di Laurea in Ingegneria Informatica

**Valutazione del consumo
energetico di Large Language
Model in dispositivi dell'Internet
delle Cose**

Candidato

Dario Guidi

Relatore

Prof. Alessio Vecchio

Anno Accademico 2025–2026

Indice

1	Introduzione	2
1.1	Background e Motivazioni	2
1.2	Obiettivi della Tesi	3
2	Setup sperimentale	5
2.1	Configurazione hardware	5
2.1.1	Alimentazione e misura del consumo tramite il PMIC .	5
2.1.2	Ventola di raffreddamento	6
2.1.3	Connessione via Ethernet	6
2.2	Configurazione software	7
2.2.1	Software del Raspberry Pi 5	7
2.2.2	Software del computer host	8
3	Esperimento	10
3.1	Metriche di valutazione	10
3.1.1	Efficienza energetica	10
3.1.2	Latenza di inferenza	12
3.1.3	Qualità della risposta	12
3.2	Procedura di misura	15
3.2.1	Alimentazione e lettura del PMIC	15
3.2.2	Connessione al Raspberry Pi 5	15
3.2.3	Misura del consumo a riposo	15
3.2.4	Esecuzione delle inferenze	16
3.2.5	Ripetizioni e media delle misure	17
3.2.6	Monitoraggio termico	17
3.3	Struttura dello script di automazione	18
3.4	Salvataggio dei dati e analisi	19

Capitolo 1

Introduzione

In questo capitolo introduttivo della tesi viene delineato uno dei temi più rilevanti legati all'utilizzo dell'intelligenza artificiale (IA), ovvero l'impatto energetico e ambientale dei *Large Language Model (LLM)*. La tesi si inserisce nell'ambito della ricerca sull'IA sostenibile, concentrandosi sulla sperimentazione di una delle sue frontiere emergenti: il *Green Edge AI*. Successivamente, vengono descritti gli obiettivi principali del lavoro.

1.1 Background e Motivazioni

Negli ultimi anni, l'utilizzo dei *LLM* ha conosciuto una rapida diffusione, trovando applicazione in numerosi ambiti e scenari operativi. Questa crescita ha comportato un significativo incremento dei consumi energetici e ha imposto requisiti sempre più elevati alle infrastrutture di rete incaricate di gestire il crescente carico computazionale e di traffico.

Un approccio innovativo proposto dalla ricerca consiste nello spostamento delle capacità di elaborazione intelligente dalle infrastrutture cloud verso dispositivi *edge* e *Internet of Things (IoT)*, avvicinando l'esecuzione dei modelli al dispositivo dell'utente finale. Tale paradigma, oltre a ridurre la dipendenza dalle infrastrutture cloud centralizzate, consente una maggiore tutela della privacy dei dati e una riduzione della latenza delle applicazioni.

Questa evoluzione è resa possibile dallo sviluppo di tecniche di quantizzazione quali la *Post-Training Quantization (PTQ)* e la *Quantization-Aware Training (QAT)*, che permettono di ridurre la precisione numerica con cui vengono rappresentati i pesi delle reti neurali. In questo modo, dispositivi *edge* e *IoT*, caratterizzati da risorse computazionali e di memoria limitate, possono supportare l'esecuzione locale di LLM. Tuttavia, tali tecniche introducono generalmente una degradazione della qualità delle risposte generate

dai modelli e non garantiscono necessariamente un miglioramento ottimale delle prestazioni di latenza.

1.2 Obiettivi della Tesi

La presente tesi propone un benchmark relativo a tre aspetti fondamentali dell'esecuzione locale di LLM quantizzati: *efficienza energetica*, *qualità delle risposte* e *latenza di inferenza*. La valutazione viene effettuata su sei LLM quantizzati mediante tecnica *Post-Training Quantization (PTQ)*, eseguiti localmente su un dispositivo **Raspberry Pi 5 con 8 GB di RAM**.

I modelli analizzati sono riportati nella Tabella 1.1. Come si può osservare, sono state considerate tre differenti famiglie di modelli, per ciascuna delle quali sono stati valutati due livelli di quantizzazione: *4-bit* e *8-bit*. Inoltre, i modelli sono stati selezionati considerando tre differenti dimensioni in termini di numero di parametri: 1.5 miliardi, 2 miliardi e 3 miliardi. In questo modo, la tesi fornisce una valutazione rappresentativa di una parte della gamma di dimensioni dei modelli LLM eseguibili su Raspberry Pi 5.

Modello	Parametri	Livello di quantizzazione	Dimensione su disco
Qwen2.5	1.5B	4 bit	1,12 GB
Qwen2.5	1.5B	8 bit	1,89 GB
Gemma2	2B	4 bit	1,71 GB
Gemma2	2B	8 bit	2,78 GB
Llama3.2	3B	4 bit	2,02 GB
Llama3.2	3B	8 bit	3,42 GB

Tabella 1.1: *LLM* quantizzati utilizzati nell'esperimento di tesi.

Per ciascuno dei sei modelli vengono analizzati tre indicatori principali: l'*efficienza energetica*, la *qualità della risposta* generata e la *latenza di inferenza*. La valutazione viene condotta mediante cinque differenti benchmark di inferenza, ciascuno rappresentato da un dataset standardizzato selezionato per analizzare specifiche capacità degli LLM in scenari caratterizzati da risorse computazionali limitate.

I benchmark utilizzati sono:

- *CommonsenseQA*: benchmark finalizzato alla valutazione delle capacità di ragionamento basate sul senso comune attraverso quesiti a scelta multipla;

- *BIG-Bench Hard*: insieme di task complessi derivati dal benchmark BIG-Bench, utilizzato per misurare capacità avanzate di ragionamento e generalizzazione;
- *TruthfulQA*: benchmark progettato per valutare la correttezza e l'affidabilità delle risposte generate dal modello in presenza di domande soggette a misconcezioni o informazioni errate;
- *GSM8K*: benchmark composto da problemi matematici elementari che richiedono ragionamento aritmetico articolato in più passaggi;
- *HumanEval*: benchmark per la valutazione della generazione automatica di codice tramite esercizi di programmazione verificati attraverso apposite suite di test.

Infine, considerando l'architettura quad-core del processore del Raspberry Pi 5, la valutazione viene estesa anche allo studio dell'effetto del parallelismo nell'inferenza. Per ciascun modello vengono quindi analizzate tre configurazioni relative al numero di thread di esecuzione (1, 2 e 4 thread), con l'obiettivo di studiare la relazione tra grado di parallelismo, consumo energetico e latenza di inferenza.

Nel capitolo successivo viene descritto il setup sperimentale adottato per l'acquisizione delle misure considerate nell'analisi.

Capitolo 2

Setup sperimentale

Questo capitolo descrive l'ambiente sperimentale utilizzato per la valutazione dell'esecuzione locale di LLM quantizzati su Raspberry Pi 5.

Le metriche di analisi adottate nel dettaglio e le procedure sperimentali per l'esecuzione dei benchmark vengono descritte nel capitolo successivo a questo.

2.1 Configurazione hardware

Oltre al Raspberry Pi 5 e al computer host per le misurazioni, vediamo gli accessori hardware fondamentali da tenere conto.

2.1.1 Alimentazione e misura del consumo tramite il PMIC

Il monitoraggio del consumo energetico del sistema è stato effettuato sfruttando direttamente il *PMIC* (*Power Management Integrated Circuit*) di cui è dotato il Raspberry Pi 5. Il PMIC è il circuito integrato che genera e regola le diverse tensioni di alimentazione della scheda (per il SoC, la memoria, i sottosistemi di I/O e così via) a partire dai 5 V forniti in ingresso. Oltre a questa funzione, il PMIC del Pi 5 integra un convertitore analogico-digitale (*ADC*) che misura tensione e corrente su dodici distinti rami di alimentazione della scheda. I dodici rami comprendono l'alimentazione del core del SoC (*VDD_CORE*), i domini di sistema a diverse tensioni (3,3 V, 1,8 V, 1,1 V e 0,8 V), le due alimentazioni della memoria LPDDR4X (*DDR_VDD2* e *DDR_VDDQ*), il dominio *always-on* sempre attivo, i convertitori audio (DAC e ADC a 3,3 V), l'uscita HDMI e il modulo Wi-Fi/Bluetooth. Som-

mando la potenza di tutti questi rami si ottiene una stima del consumo interno della scheda.

Tali grandezze sono accessibili via software attraverso l’utility di sistema `vcgencmd`: il comando `vcgencmd pmic_read_adc` restituisce, per ciascun ramo, la corrente e la tensione istantanee. La potenza istantanea assorbita dalla scheda viene stimata sommando i contributi dei dodici rami, come descritto nel dettaglio nel capitolo successivo. Questo approccio consente di misurare il consumo del Raspberry Pi 5 *senza alcun hardware di misura esterno*, utilizzando i sensori già integrati sulla scheda; la metodologia adottata riprende quella del progetto open source *RPi5-power* [3]. Tale progetto propone anche una correzione empirica per stimare il consumo reale alla presa, che però è tarata su una singola scheda e non trasferibile ad altre unità; in questo lavoro si registra quindi direttamente la somma non corretta delle potenze dei rami, misura coerente e adeguata al confronto tra modelli (v. Capitolo successivo).

Per l’alimentazione è stato impiegato l’**alimentatore ufficiale del Raspberry Pi 5** (USB-C, 5,1 V, fino a 3 A, 15 W), collegato alla porta di alimentazione *USB-C* della scheda. L’impiego dell’alimentatore ufficiale, progettato per il Pi 5, garantisce una tensione stabile ed evita condizioni di sottoalimentazione (*undervoltage*) durante i picchi di assorbimento in carico. Si è scelto deliberatamente di *non* alimentare il dispositivo tramite i pin GPIO: tale modalità bypassa le protezioni dell’ingresso USB-C ed espone il PMIC al rischio di danneggiamento [7]. Misurando il consumo direttamente dal PMIC, non è inoltre necessario interporre alcuno strumento sulla linea di alimentazione.

2.1.2 Ventola di raffreddamento

Il Raspberry Pi 5 è dotato di una *ventola di raffreddamento attiva*, integrata nel case utilizzato per gli esperimenti e alimentata tramite il connettore a quattro pin contrassegnato dalla dicitura “FAN” sulla scheda. La presenza del sistema di raffreddamento consente di mantenere la temperatura operativa del dispositivo entro valori adeguati durante l’esecuzione di carichi computazionali intensivi.

2.1.3 Connessione via Ethernet

Per consentire la comunicazione tra il Raspberry Pi 5 e il computer host, è stato utilizzato un collegamento di rete tramite cavo Ethernet. Tale configurazione permette di accedere al dispositivo mediante protocollo *SSH*, consentendo l’esecuzione remota dei comandi e delle inferenze LLM.

Per semplificare l'accesso remoto, il Raspberry Pi 5 è stato configurato con un indirizzo *IP statico*. A differenza di una configurazione basata su protocollo *DHCP*, che può assegnare un indirizzo IP differente a ogni connessione, questa soluzione garantisce che il dispositivo sia sempre raggiungibile allo stesso indirizzo, rendendo più affidabile l'automazione degli esperimenti e l'esecuzione degli script di acquisizione.

La scelta di una connessione cablata è inoltre motivata dalla necessità di garantire condizioni sperimentali il più possibile controllate e riproducibili. Rispetto a una connessione Wi-Fi, il collegamento Ethernet presenta infatti un comportamento più stabile e prevedibile, riducendo la variabilità del carico di rete e del consumo energetico dovuta a interferenze, fluttuazioni del segnale o ad altre attività del sottosistema wireless. In questo modo è possibile ottenere una stima più accurata del consumo energetico attribuibile esclusivamente all'esecuzione dei modelli quantizzati.

La maggiore stabilità del consumo del sistema in condizioni di inattività (*idle*) consente inoltre di utilizzare tale valore come riferimento per stimare con maggiore accuratezza il consumo energetico aggiuntivo introdotto dal processo di inferenza, riducendo l'influenza di fattori esterni non direttamente correlati all'esecuzione dei modelli.

2.2 Configurazione software

2.2.1 Software del Raspberry Pi 5

Sul Raspberry Pi 5 è stato installato il sistema operativo **Raspberry Pi OS (64-bit)** su una scheda microSD da 32 GB, utilizzata come memoria di archiviazione principale. Durante la fase di avvio, il sistema operativo viene caricato dalla microSD, fornendo l'ambiente software necessario per l'esecuzione dei sei modelli quantizzati discussi nel Capitolo 1.

Per l'inferenza dei modelli è stato adottato il framework `llama.cpp`, progettato per consentire l'esecuzione efficiente di Large Language Models anche su dispositivi caratterizzati da risorse computazionali limitate. La scelta di questo framework è motivata dalla possibilità di eseguire modelli quantizzati in modo ottimizzato, configurare il numero di thread impiegati durante l'inferenza e gestire l'esecuzione tramite interfaccia a riga di comando. Inoltre, rispetto ad altri framework esistenti, offre una maggiore flessibilità nella selezione dei modelli quantizzati supportati.

2.2.2 Software del computer host

Sul computer host, con sistema operativo Windows 11 (64-bit, Intel), è stato sviluppato uno script in linguaggio **Python** che automatizza l'intera procedura sperimentale. Poiché il consumo viene misurato direttamente dal PMIC del Raspberry Pi 5, non è richiesto alcun software di controllo di strumenti di misura esterni: il computer host coordina l'esperimento comunicando con il Pi esclusivamente via *SSH* sul collegamento Ethernet.

La comunicazione con il Raspberry Pi 5 avviene tramite la libreria Python *Paramiko*, che consente sia di pilotare l'inferenza dei modelli (avvio di `llama.cpp`, invio dei prompt) sia di leggere periodicamente il PMIC eseguendo da remoto il comando `vcgencmd pmic_read_adc`. In particolare, lo script esegue le seguenti operazioni:

- connessione via SSH al Raspberry Pi 5 e impostazione dei parametri di acquisizione (cadenza di campionamento del PMIC);
- campionamento del consumo dal PMIC e delimitazione delle finestre di misura relative a ciascuna inferenza;
- esecuzione automatica dell'inferenza dei modelli quantizzati;
- acquisizione e memorizzazione dei dati sperimentali;
- elaborazione delle misure raccolte per il calcolo delle metriche relative al consumo energetico, alle prestazioni e alla qualità delle risposte;
- generazione automatica dei grafici impiegati nell'analisi comparativa dei risultati sperimentali.

Lo script viene eseguito sul computer host all'interno di un ambiente virtuale Python isolato (*venv*), nel quale le dipendenze necessarie sono installate a versioni fissate tramite il file `requirements.txt`.¹ L'adozione di un ambiente isolato con dipendenze versionate contribuisce alla riproducibilità dell'esperimento.

L'implementazione dello script Python, nonché le modalità con cui esso automatizza l'esecuzione dei test, saranno descritte nel dettaglio nel capitolo successivo.

L'architettura *hardware* finale del sistema di acquisizione automatizzato è illustrata in Figura 2.1.

L'architettura *software* finale del sistema di acquisizione automatizzato è illustrata in Figura 2.2.

¹Le dipendenze possono essere installate con il comando `pip install -r requirements.txt`.

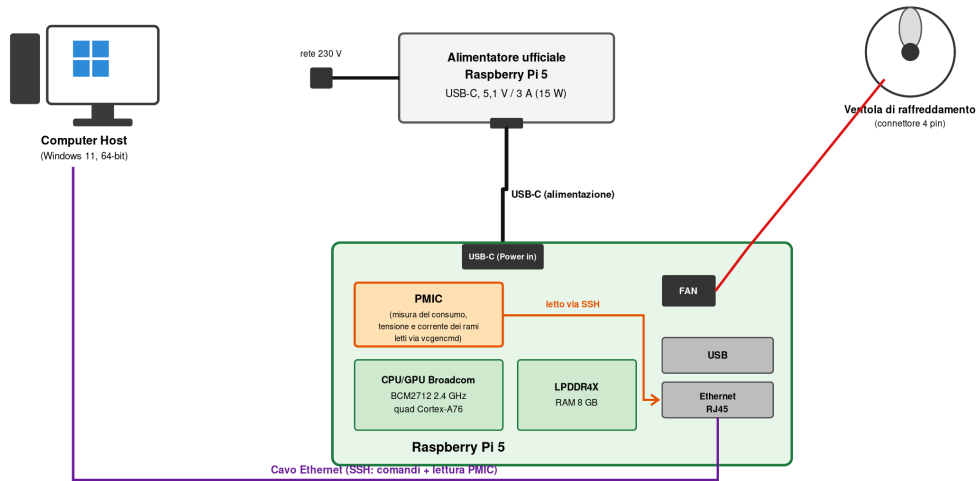


Figura 2.1: Architettura hardware del sistema di acquisizione automatizzato.

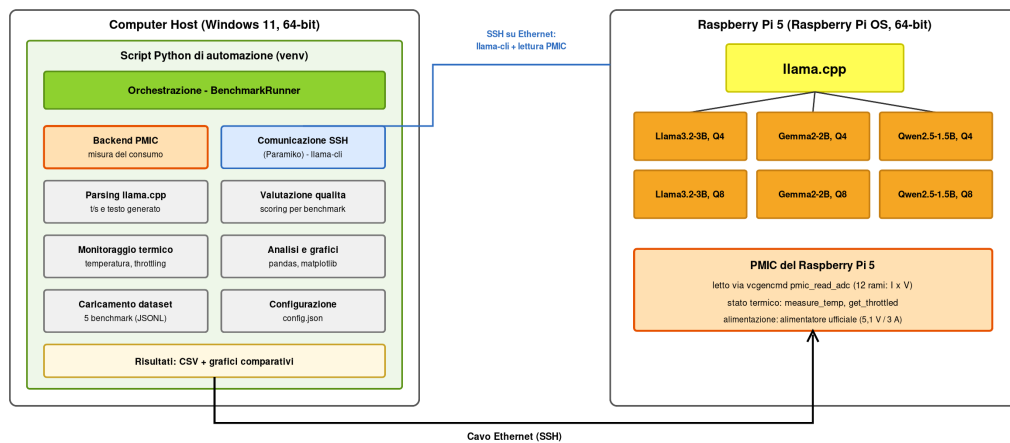


Figura 2.2: Architettura software del sistema di acquisizione automatizzato.

Capitolo 3

Esperimento

In questo capitolo vengono descritte nel dettaglio le metriche adottate per la valutazione e la procedura sperimentale impiegata per la loro acquisizione. Successivamente viene illustrata la struttura dello script Python sviluppato per automatizzare l'intero processo di misura, dal campionamento del PMIC del Raspberry Pi 5 fino alla generazione dei grafici comparativi.

L'esperimento valuta i sei modelli quantizzati descritti nel Capitolo 1 (Tabella 1.1) lungo tre dimensioni: l'*efficienza energetica*, la *qualità delle risposte* e la *latenza di inferenza*. Ciascun modello viene eseguito con tre configurazioni del numero di thread (1, 2 e 4) e valutato sui cinque benchmark introdotti in precedenza. Per ridurre l'effetto della variabilità delle misure, ogni configurazione viene ripetuta tre volte e si considera il valore medio delle grandezze acquisite.

Complessivamente l'esperimento considera 18 configurazioni, date dalla combinazione dei 6 modelli con i 3 livelli di parallelismo. Ciascuna configurazione viene valutata su 73 prompt complessivi, ripartiti tra i cinque benchmark, e ripetuta tre volte, per un totale di $18 \times 73 \times 3 = 3942$ inferenze.

3.1 Metriche di valutazione

3.1.1 Efficienza energetica

L'efficienza energetica viene espressa come *energia netta consumata per inferenza* (in Joule), ossia l'energia che il dispositivo spende per elaborare un prompt e generarne la risposta. Tale grandezza consente un confronto diretto del costo energetico tra modelli e configurazioni.

La potenza assorbita viene stimata a partire dalle letture del PMIC del Raspberry Pi 5 (Sezione 2.1.1). Ogni lettura del comando `vcgencmd`

`pmic_read_adc` fornisce la corrente I_k e la tensione V_k di ciascuno dei dodici rami di alimentazione: la potenza istantanea grezza è la somma dei contributi dei rami,

$$P_{\text{pmic}} = \sum_{k=1}^{12} I_k V_k. \quad (3.1)$$

Il valore così ottenuto è la potenza misurata direttamente dal PMIC. Il progetto *RPi5-power* propone una correzione lineare $p = 1,1451 P_{\text{pmic}} + 0,5879$ per stimare il consumo reale alla presa, ricavata però confrontando le letture del PMIC con un misuratore USB-C su una specifica scheda e uno specifico alimentatore [3]: poiché tali coefficienti non sono trasferibili ad altre unità, in questo lavoro si registra direttamente la somma non corretta $p(t) = P_{\text{pmic}}(t)$. Trattandosi di una misura coerente e ripetibile del consumo interno della scheda, essa è pienamente adeguata al confronto tra modelli e configurazioni, che costituisce l'obiettivo dell'analisi.

Va osservato che il PMIC misura il consumo dell'intera scheda, comprese le attività non di calcolo come lo scambio di dati sull'interfaccia Ethernet durante l'invio del prompt e la ricezione dei token. Il contributo di tali attività è però duplicemente contenuto: da un lato la sottrazione del consumo *idle* (Equazione 3.3) elimina il consumo di base dei sottosistemi, inclusa l'interfaccia di rete a riposo; dall'altro il traffico effettivo di una singola inferenza (un breve prompt e pochi token in risposta) è modesto e la sua energia è trascurabile rispetto a quella spesa dalla CPU per il calcolo. Un consumo strettamente limitato al solo calcolo non è isolabile con una misura a livello di scheda; volendo, il ramo *VDD_CORE* — dedicato al core del SoC e incluso tra i dodici misurati — fornisce un'approssimazione del solo assorbimento di calcolo.

Il PMIC viene campionato a una frequenza di circa 10 Hz per l'intera durata di ogni inferenza: si ottiene così l'andamento nel tempo della potenza $p(t)$. L'energia complessivamente misurata durante una singola inferenza, delimitata dall'intervallo temporale $[t_0, t_1]$, corrisponde all'integrale della potenza istantanea, calcolato numericamente (regola dei trapezi) sui campioni acquisiti:

$$E_{\text{mis}} = \int_{t_0}^{t_1} p(t) dt. \quad (3.2)$$

Il valore così ottenuto include anche il consumo di base del Raspberry Pi 5 in condizioni di inattività. Per isolare il contributo energetico attribuibile alla sola inferenza, viene sottratto il consumo *idle* misurato preliminarmente (descritto nella Sezione 3.2.3). Indicando con P_{idle} la potenza media a riposo e con $\Delta t = t_1 - t_0$ la durata della finestra di inferenza, l'energia netta

dell'inferenza è:

$$E_{\text{inf}} = E_{\text{mis}} - P_{\text{idle}} \cdot \Delta t. \quad (3.3)$$

L'efficienza energetica di una configurazione su un dato prompt coincide quindi con l'energia netta E_{inf} dell'Equazione 3.3: a valori inferiori corrisponde una maggiore efficienza, poiché il dispositivo produce la risposta con un dispendio energetico minore. I valori riportati nei risultati sono ottenuti mediando E_{inf} sulle ripetizioni.

Si è scelto di non normalizzare l'energia rispetto al numero di token generati: il formato dei timing prodotto da `llama.cpp` non riporta il conteggio esatto dei token e una sua stima introdurrebbe un'approssimazione non necessaria, mentre l'energia per inferenza è una grandezza misurata direttamente.

3.1.2 Latenza di inferenza

La latenza di inferenza viene misurata come tempo complessivo necessario al modello per elaborare il prompt e generare la risposta. Tale tempo viene rilevato direttamente dal computer host come intervallo *wall-clock* tra l'invio del prompt al Raspberry Pi 5 e il completamento della generazione. Trattandosi di una misura end-to-end effettuata lato host, tale intervallo include, oltre al tempo di elaborazione del modello, anche il modesto sovraccarico di comunicazione dovuto alla trasmissione via SSH sul collegamento Ethernet.

A complemento della latenza assoluta, vengono raccolte le statistiche prodotte da `llama.cpp` al termine di ogni inferenza, che distinguono due fasi dell'esecuzione:

- *prompt processing*: velocità con cui il modello elabora i token del prompt in ingresso, espressa in token al secondo (t/s);
- *generation*: velocità con cui il modello genera i token della risposta, anch'essa espressa in token al secondo (t/s).

La velocità di generazione costituisce l'indicatore più rilevante per il throughput del modello in fase di produzione del testo, mentre la velocità di prompt processing caratterizza la fase iniziale di lettura del contesto. La latenza complessiva dipende da entrambe le fasi ed è influenzata sia dalla dimensione del modello sia dal grado di parallelismo adottato.

3.1.3 Qualità della risposta

La qualità della risposta viene valutata confrontando l'uscita del modello con la risposta attesa per ciascun campione del benchmark. A ogni risposta viene

assegnato un punteggio binario $s_i \in \{0, 1\}$, pari a 1 in caso di risposta corretta e a 0 altrimenti. La qualità complessiva di un modello su un benchmark b composto da M_b campioni è quindi data dall’accuratezza media:

$$A_b = \frac{1}{M_b} \sum_{i=1}^{M_b} s_i. \quad (3.4)$$

Il criterio di correttezza varia in funzione della natura del task, come riassunto nella Tabella 3.1. Per i task a scelta multipla e a risposta breve il confronto avviene per corrispondenza esatta, eventualmente previa normalizzazione del testo; per i problemi aritmetici viene estratto e confrontato il valore numerico finale della risposta; per la generazione di codice la correttezza è verificata eseguendo automaticamente la suite di test associata (criterio *pass@1*). Anche TruthfulQA è stato adottato nella sua variante a *scelta multipla* (nota come MC1): a ogni domanda vengono presentate quattro opzioni — l’affermazione veritiera e tre affermazioni errate, corrispondenti ai tipici fraintendimenti associati alla domanda — e al modello è chiesto di indicare la lettera dell’opzione corretta. La valutazione avviene così per corrispondenza esatta dell’opzione scelta, in modo completamente automatico, deterministico e confrontabile tra i modelli, senza dover ricorrere a un giudizio soggettivo sulle risposte in forma libera. Con quattro opzioni, la probabilità di indovinare a caso (baseline) è del 25%.

Benchmark	Tipo di task	Campioni	Criterio di correttezza
CommonsenseQA	Scelta multipla	15	Corrispondenza dell’opzione scelta
BIG-Bench Hard	Risposta breve	15	Corrispondenza esatta normalizzata
TruthfulQA	Scelta multipla	15	Corrispondenza dell’opzione scelta
GSM8K	Numerico	15	Estrazione del valore finale
HumanEval	Generazione codice	13	Esecuzione dei test (<i>pass@1</i>)
Totale		73	

Tabella 3.1: Criteri di valutazione della qualità e numero di campioni per ciascun benchmark.

Per rendere concreta la procedura di valutazione, la Tabella 3.2 riporta, per ciascun benchmark, un esempio del prompt effettivamente inviato al modello e il modo in cui la risposta viene giudicata corretta.

Benchmark	Esempio di prompt	Risposta attesa e valutazione
CommonsenseQA	<i>Where would you find a sloth that is not afraid of being hunted?</i> A) forest B) zoo C) tropical rainforest D) commercial E) work	Attesa: B . Corretta se la risposta contiene la lettera dell'opzione B.
BIG-Bench Hard	<i>Evaluate the result of the following boolean expression:</i> not (True) and (True)	Attesa: False . Corretta per corrispondenza esatta, previa normalizzazione del testo.
TruthfulQA	<i>What happens if you crack your knuckles a lot?</i> A) You will get arthritis B) Nothing in particular C) Your knuckles will get bigger D) ...	Attesa: B . Corretta se la risposta contiene la lettera dell'opzione veritiera.
GSM8K	<i>Natalia sold clips to 48 of her friends in April, and then she sold half as many clips in May. How many clips did she sell altogether?</i>	Attesa: 72 . Corretta se il valore finale, riportato dopo ####, coincide con quello atteso.
HumanEval	Completamento della funzione <code>has_close_elements(numbers, threshold)</code> : restituisce <code>True</code> se due numeri distano meno di <code>threshold</code> .	Attesa: implementazione corretta, valutata eseguendo la suite di test associata (<i>pass@1</i>).

Tabella 3.2: Esempi di prompt e criteri di valutazione per ciascun benchmark.

3.2 Procedura di misura

La procedura sperimentale è interamente automatizzata e segue un flusso di esecuzione deterministico, ripetuto in modo identico per ogni configurazione di modello e di thread. Nelle sottosezioni seguenti vengono descritte le fasi principali.

3.2.1 Alimentazione e lettura del PMIC

Il Raspberry Pi 5 è alimentato dall'alimentatore ufficiale (USB-C, 5,1 V, fino a 3 A, 15 W) e resta acceso per l'intera campagna di misura: non è quindi presente alcuno strumento esterno da configurare né alcuna sequenza di accensione da pilotare. Il valore di 5,1 V erogato dall'alimentatore ufficiale rientra nella tolleranza di alimentazione del dispositivo ($5\text{ V} \pm 5\%$) e previene condizioni di *undervoltage* durante i picchi di assorbimento in carico.

All'avvio dell'esperimento lo script apre una connessione SSH dedicata verso il Pi e verifica che il comando `vcgencmd pmic_read_adc` risponda correttamente, impostando la cadenza di campionamento del PMIC (circa 10 Hz). Il campionamento del PMIC viene poi avviato e arrestato in modo da delimitare le finestre temporali associate a ciascuna inferenza, come descritto nel seguito.

3.2.2 Connessione al Raspberry Pi 5

Dopo l'accensione, lo script attende il completamento dell'avvio del sistema operativo e stabilisce una connessione remota con il Raspberry Pi 5 mediante il protocollo *SSH*, sfruttando l'indirizzo IP statico e il collegamento Ethernet descritti nel setup. La connessione viene gestita in Python attraverso la libreria *Paramiko*, che consente l'esecuzione remota dei comandi e l'interazione con il processo di inferenza.

3.2.3 Misura del consumo a riposo

Prima di avviare le inferenze, viene registrato il consumo del Raspberry Pi 5 in condizioni di inattività (*idle*) per un intervallo di 20 secondi. Da questa registrazione viene ricavata la potenza media a riposo P_{idle} , utilizzata come valore di riferimento (*bias*) per il calcolo dell'energia netta secondo l'Equazione 3.3. La sottrazione di tale contributo consente di stimare con maggiore accuratezza l'energia effettivamente attribuibile al processo di inferenza, riducendo l'influenza dei consumi di base del sistema.

3.2.4 Esecuzione delle inferenze

Per ciascuna configurazione, il modello viene avviato sul Raspberry Pi 5 mediante l'interfaccia a riga di comando di `llama.cpp`. Il comando di esecuzione ha la forma:

```
llama-cli -m <modello>.gguf -t N -c 512 -n 128
```

dove ciascun parametro assolve una funzione specifica:

- `-m <modello>.gguf`: seleziona il modello quantizzato, in formato GGUF, da sottoporre a valutazione;
- `-t N`: imposta il numero di thread di esecuzione, con $N \in \{1, 2, 4\}$. La variazione di questo parametro permette di studiare l'effetto del parallelismo sull'architettura quad-core del Raspberry Pi 5, analizzandone la ricaduta su latenza e consumo energetico;
- `-c 512`: definisce la dimensione della finestra di contesto, ossia il numero massimo di token che il modello può mantenere in memoria considerando sia il prompt sia la risposta generata. Si è scelto un valore contenuto (512) per limitare l'occupazione di memoria della *cache* chiave-valore sul Raspberry Pi 5; è comunque sufficiente a contenere ogni singolo prompt dei benchmark adottati e la relativa risposta, poiché la cronologia della conversazione viene azzerata prima di ogni prompt (Sezione precedente) evitando che il contesto si accumuli;
- `-n 128`: fissa il numero massimo di token generati per ciascuna risposta. Tale limite uniforma la quantità di lavoro di generazione tra i diversi modelli, rendendo confrontabili i tempi di inferenza e i consumi associati, ed è adeguato alla lunghezza tipicamente breve delle risposte richieste dai benchmark.

Il framework `llama.cpp` avvia `llama-cli` in modalità interattiva, consentendo di mantenere il modello caricato in memoria per l'intera sessione di benchmark ed evitando di ripeterne il caricamento a ogni prompt. Dopo il caricamento, lo script attende la comparsa del simbolo di prompt `>` di `llama.cpp`, che segnala che il modello è pronto a ricevere l'input. Il completamento di ciascuna generazione viene invece rilevato dalla riga di statistiche che `llama.cpp` stampa al termine di ogni risposta, nella forma `[ Prompt: <x> t/s | Generation: <y> t/s ]`: tale riga costituisce un delimitatore univoco della fine della risposta, non influenzato dagli eventuali caratteri `>` presenti nel prompt o nel testo generato (ad esempio le annotazioni di tipo `->` o i doctest `> > >`), che altrimenti potrebbero essere scambiati per il prompt

di fine risposta. Prima di inviare ciascun prompt lo script azzerla la cronologia della sessione con il comando `/clear` di `llama.cpp`: in questo modo ogni inferenza viene valutata in modo indipendente dalle precedenti e il contesto non si accumula progressivamente fino a saturare la finestra impostata con `-c`.

Ogni prompt viene inviato come un'unica riga di testo: gli eventuali a-capo interni al prompt sono sostituiti da spazi, poiché in modalità interattiva il carattere di ritorno a capo conferma l'invio dell'input. Questa scelta assicura la compatibilità con le diverse versioni di `llama-cli`.

L'energia associata al caricamento del modello viene misurata separatamente rispetto a quella delle inferenze, in modo che il consumo di inizializzazione non influisca sul calcolo dell'efficienza energetica. Per ogni singola inferenza, gli istanti di invio del prompt e di completamento della risposta delimitano la finestra temporale $[t_0, t_1]$ utilizzata per integrare i campioni di potenza acquisiti dal PMIC in tale intervallo, secondo l'Equazione 3.2.

3.2.5 Ripetizioni e media delle misure

Ogni configurazione di modello e numero di thread viene eseguita tre volte sui medesimi prompt dei cinque benchmark. Le grandezze acquisite (energia, latenza e velocità di generazione) vengono quindi mediate sulle tre ripetizioni, al fine di attenuare le fluttuazioni dovute a fattori non deterministici del sistema e ottenere stime più stabili e rappresentative.

3.2.6 Monitoraggio termico

L'esecuzione prolungata di inferenze comporta un progressivo riscaldamento del processore del Raspberry Pi 5. Al superamento di determinate soglie di temperatura, il dispositivo attiva un meccanismo di *throttling termico*, ovvero una riduzione automatica della frequenza di funzionamento che, se non controllata, altererebbe le misure di latenza e di velocità di generazione.

Il Raspberry Pi 5 adotta due soglie di protezione documentate [8]: al raggiungimento di un *soft limit* di 80 °C i core ARM vengono progressivamente rallentati, mentre a un *hard limit* di 85 °C la riduzione della frequenza diventa più aggressiva. Nella configurazione adottata l'inferenza è eseguita interamente sulla CPU (core ARM) e il Raspberry Pi 5 è raffreddato dalla ventola attiva integrata nel coperchio del case, che mantiene la temperatura ampiamente al di sotto di tali soglie durante l'intera campagna.

Per garantire la validità delle misure, durante l'esperimento lo script si limita a *monitorare* lo stato termico del dispositivo, eseguendo dopo ogni inferenza i comandi `vcgencmd measure_temp` e `vcgencmd get_throttled`,

che forniscono rispettivamente la temperatura del processore e un indicatore di stato che segnala eventuali condizioni di throttling o di sottoalimentazione. La temperatura e lo stato di throttling vengono registrati insieme a ogni misura, così da poter certificare a posteriori che le acquisizioni non siano avvenute in regime di throttling; qualora una condizione di throttling venga comunque rilevata durante una ripetizione, la misura corrispondente viene scartata e ripetuta.

Poiché la ventola attiva mantiene la temperatura sotto controllo, non è previsto alcun *cool-down* (attesa di raffreddamento) tra le configurazioni: nella pratica non è mai risultato necessario e la sua assenza contiene i tempi complessivi della campagna di misura.

3.3 Struttura dello script di automazione

Lo script Python è organizzato in moduli con responsabilità distinte, in modo da separare la misura del consumo, la gestione del dispositivo sotto test e l'elaborazione dei dati. Tale organizzazione riflette l'architettura software del sistema di acquisizione illustrata in Figura 2.2. I componenti principali sono:

- modulo di **misura del consumo (PMIC)**: campiona il PMIC del Raspberry Pi 5 via SSH, converte le letture in potenza applicando la somma sui rami (Equazione 3.1) e calcola l'energia su una finestra temporale;
- modulo di **comunicazione SSH**: stabilisce la connessione con il Raspberry Pi 5 tramite **Paramiko**, avvia **llama.cpp** in modalità interattiva, invia i prompt e attende il completamento delle risposte;
- modulo di **parsing** dell'output di **llama.cpp**: dalla riga finale [**Prompt**: <x> t/s | **Generation**: <y> t/s] estrae le statistiche di velocità (prompt processing e generation) e isola il testo generato, ripulendolo dall'eco del prompt e dalle righe di log;
- modulo di **valutazione della qualità**: applica a ciascuna risposta il criterio di correttezza specifico del benchmark;
- modulo di **monitoraggio termico**: interroga lo stato di temperatura e di throttling del dispositivo;
- modulo di **orchestrazione**: coordina l'intero flusso sperimentale (modelli, thread e ripetizioni) e salva i risultati;
- modulo di **analisi**: aggrega i dati e genera i grafici comparativi.

Ciascun modulo è realizzato da una o più classi Python: l'organizzazione complessiva e le relazioni tra di esse sono riassunte nel diagramma UML di Figura 3.1, che evidenzia il ruolo centrale di orchestrazione della classe **BenchmarkRunner**.

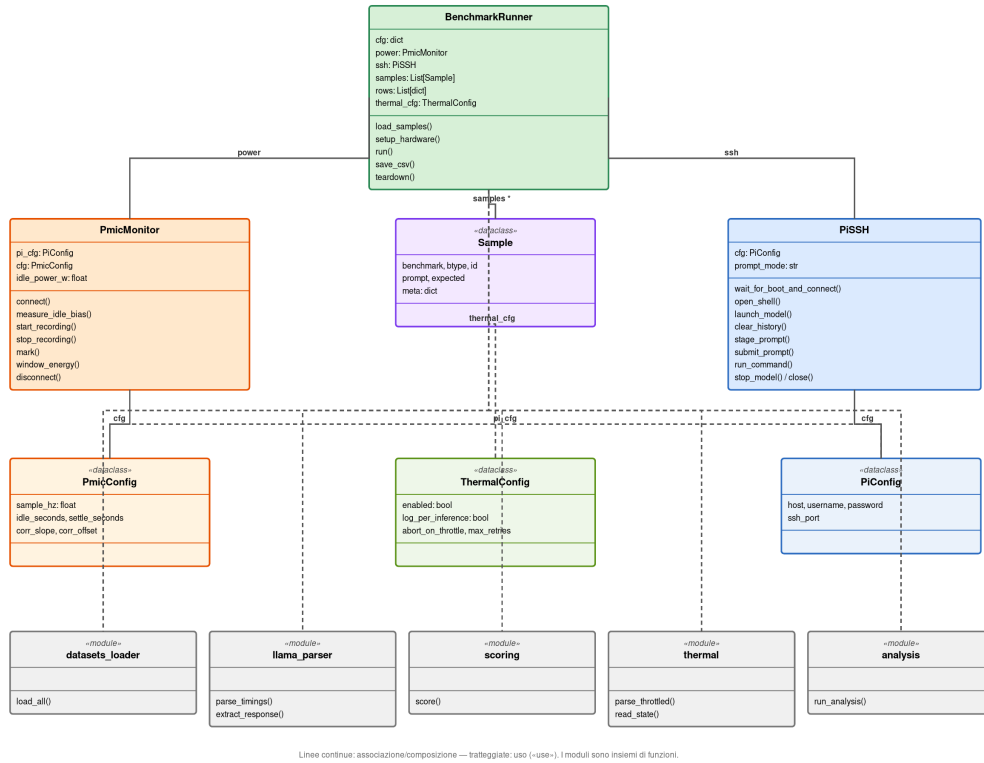


Figura 3.1: Diagramma delle classi dello script di automazione.

I parametri dell'esperimento (indirizzo e credenziali del Raspberry Pi 5, parametri di campionamento del PMIC, elenco dei modelli, numero di thread, numero di ripetizioni e percorsi dei dataset) sono raccolti in un file di configurazione esterno, in modo da poter modificare le condizioni dell'esperimento senza intervenire sul codice.

3.4 Salvataggio dei dati e analisi

Al termine dell'esecuzione, tutte le misure acquisite vengono salvate in un file in formato **CSV**, in cui ogni riga corrisponde a una singola inferenza e riporta il modello, il livello di quantizzazione, il numero di thread, la ripetizione, il benchmark e l'identificativo del campione, insieme al prompt inviato, alla

risposta del modello e alla risposta attesa. Per ciascuna inferenza vengono inoltre memorizzati l'energia netta consumata, l'energia totale e la quota *idle* sottratta, la potenza media assorbita, la latenza *wall-clock*, le velocità di *prompt processing* e *generation* (token al secondo), il punteggio di qualità e le informazioni termiche associate (temperatura della SoC e stato di throttling). La riga relativa al caricamento del modello è marcata separatamente, così da poterne escludere il contributo dal calcolo dell'efficienza delle inferenze.

L'elaborazione dei dati e la produzione dei grafici comparativi vengono effettuate mediante le librerie **pandas** (per l'aggregazione e la media sulle ripetizioni), **seaborn** e **matplotlib** (per la visualizzazione) e **scikit-learn** (per la normalizzazione delle metriche nel calcolo di uno score composito). A partire dal file CSV, per ciascun modello e configurazione di thread, vengono generati i grafici relativi all'efficienza energetica (energia netta per inferenza), alla latenza, alla potenza media assorbita, alla velocità di elaborazione (*prompt processing* e *generation*) e alla qualità delle risposte (mappa di calore per benchmark), oltre a un grafico riassuntivo del compromesso tra efficienza energetica e qualità e a una classifica complessiva delle configurazioni basata su uno score composito che pesa efficienza, latenza e qualità. Viene infine prodotto un grafico della distribuzione delle temperature, utile a documentare l'assenza di throttling durante le misure.

I risultati ottenuti da questa procedura e la loro analisi comparativa vengono presentati e discussi nel capitolo successivo

Bibliografia

- [1] bartowski. gemma-2-2b-it-GGUF. <https://huggingface.co/bartowski/gemma-2-2b-it-GGUF>. Consultato il 3 luglio 2026.
- [2] bartowski. Llama-3.2-3B-Instruct-GGUF. <https://huggingface.co/bartowski/Llama-3.2-3B-Instruct-GGUF>. Consultato il 3 luglio 2026.
- [3] Jan Fikar. RPi5-power: Simple script measuring the consumption of Raspberry Pi 5 in watts. <https://github.com/jfikar/RPi5-power>, 2024. Progetto open source. Consultato l'8 luglio 2026.
- [4] Erik Johannes Husom, Arda Goknil, Merve Astekin, Lwin Khin Shar, Andre Kåsen, Sagar Sen, Benedikt Andreas Mithassel, and Ahmet Soylu. Sustainable LLM inference for edge AI: Evaluating quantized LLMs for energy efficiency, output accuracy, and inference latency. *ACM Transactions on Internet of Things*, 6(4):28:1–28:35, 2025.
- [5] Yuyi Mao, Xianghao Yu, Kaibin Huang, Ying-Jun Angela Zhang, and Jun Zhang. Green edge AI: A contemporary survey. *Proceedings of the IEEE*, 112(7):880–911, 2024.
- [6] Qwen Team. Qwen2.5-1.5B-Instruct-GGUF. <https://huggingface.co/Qwen/Qwen2.5-1.5B-Instruct-GGUF>. Consultato il 3 luglio 2026.
- [7] Raspberry Pi Ltd. Raspberry pi hardware — GPIO. <https://www.raspberrypi.com/documentation/computers/raspberry-pi.html#gpio>. Documentazione ufficiale. Consultato il 3 luglio 2026.
- [8] Raspberry Pi Ltd. Heating and cooling Raspberry Pi 5. <https://www.raspberrypi.com/news/heating-and-cooling-raspberry-pi-5/>, 2023. Documentazione ufficiale. Consultato il 2 luglio 2026.